

Pattern Matching and Machine Learning for Audio Signal Processing

Prof. Dr. Frank Kurth

Prof. Dr. Michael Clausen

INSTITUTE OF COMPUTER SCIENCE, UNIVERSITY OF BONN

Chapter 4: Audio Identification

Chapter 4: Audio Identification	2
Goals of Chapter 4	2
Information retrieval: background *	3
Content-based audio retrieval *	4
Query-by-example *	5
Audio identification: the scenario *	6
Client-server model of audio ID.	7
Design of audio fingerprints	8
Illustration of peak selection	9
Fingerprint matching: basic idea	10
Illustration of fingerprint matching	11
Formalization of matching procedure	12
Indexing: preliminary remarks	13
Inverted lists: preliminary remarks	14
Formalization of index-based retrieval	15
Illustration of indexed-based retrieval	16
Computation of the matching function	17
Complexity of computing matching functions	18
Gain of indexed-based computation	19
Indexed-based audio identification	20
Improved hashing via pairs of peaks	21
A new feature representation	22
Audio identification revisited	23
Storage requirements of the new approach	24
Speed-up of the new approach	25
Robustness of the new approach (1)	26
Robustness of the new approach (2)	27
Further applications *	28
References *	29

Goals of Chapter 4

- to provide general background info on information retrieval
- to specify content-based audio retrieval tasks
- to discuss audio fingerprints based on spectral peaks
- to introduce retrieval techniques: indexing and inverted lists
- to present Shazam's index-based audio ID technique

Information retrieval: background *

- **Information retrieval** (IR) is devoted to the task of organizing useful information and making it accessible.
- An **information retrieval process** begins when a user specifies a **query**.
- An **information retrieval system** should then deliver from a data collection all **documents** or **items** that are somehow related to the query.
- For example, in the case of a **typical web search**, the query may consist of a text string of words and the search engine should deliver pointers to all text documents containing the specified words.
- Meanwhile, a good deal of the digital content is **non-textual**:
audio, images, video, multimedia documents, ...
- Let us focus upon the **music domain**. Nowadays, music lovers enjoy ubiquitous access to huge music collections containing audio recordings, digitized images of sheet music, album covers, and an increasing number of video clips.
- Such huge music collections require retrieval strategies that allow users to explore such collections in a convenient and enjoyable way.
- Most of the available services for **music recommendation** rely on **editorial metadata** and **tags** describing musical style, genre, mood, musical key, or tempo, among others.

Content-based audio retrieval *

- While **text-based retrieval systems** can be very powerful, they require the audio material to be enriched with suitable metadata—an assumption that is often not valid.
- Furthermore, not all retrieval scenarios can be handled by a purely text-based approach. Here are some **problematic examples**:
 - ◆ How should a retrieval system be designed, if the user's query consists of a short excerpt of a CD recording or a melody sung or whistled into a microphone?
 - ◆ How can a user be satisfied if he or she looks for music with a specific rhythmic pattern or harmonic progression which have not been annotated?
- To handle such scenarios, one requires **content-based retrieval systems**. Such systems only make use of the **raw music data**, rather than relying on manually generated metadata.

Query-by-example *

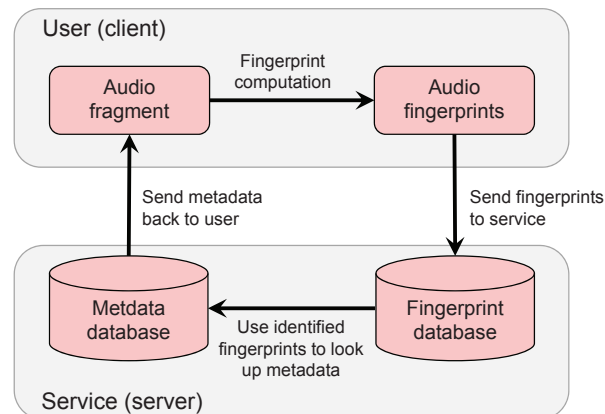
- In this and the next chapter, we present two content-based retrieval strategies that follow the **query-by-example** paradigm: given a music representation or a fragment of it (used as query or example), the task is to automatically retrieve documents from a music collection containing parts or aspects that are **similar** to the query.
- Such strategies can be loosely classified according to their **specificity**, which refers to the degree of similarity between the query and the database documents. **High specificity** is related to a strict notion of similarity, whereas **low specificity** refers to a rather vague one.
- In **Chapter 4** we discuss **audio identification**, an IR task at a high specificity level, whereas **Chapter 5** is concerned with **audio matching**, an IR task at a lower specificity level.

Audio identification: the scenario *

- Suppose you hear a song in a restaurant, a bar, or a shopping mall, and you want to know, e.g., the song's title or the name of the performer.
- Recent **music discovery services** help users in such situations by identifying the audio recording and delivering suitable content information.
- In this chapter we will discuss details of such a service, which is based on a **client-server model**. We briefly describe the components of this model:
- A user, called **client**, records a short audio fragment of the unknown song using a smartphone. With the help of an **app**, the smartphone converts the audio fragment into a compact **fingerprint** representation, which is then transmitted to the identification service, called the **server**.
- The server hosts various resources including a **fingerprint database** that covers all recordings to be identified, as well as a **metadata database** that contains content information linked to these recordings.
- The server receives the query fingerprints sent by the client and compares them with the fingerprints contained in the database. This step is typically realized by an efficient database look-up supported by suitable **index structures**.
- In the case of a successful identification, the server retrieves the content information linked to the identified fingerprints and sends back the desired metadata to the client.

Client-server model of audio ID

- The client-server model of audio identification is illustrated in the following figure.



Design of audio fingerprints

- Given an audio signal as a sampled waveform $x : \mathbb{Z} \rightarrow \mathbb{R}$, the computation of a corresponding audio fingerprint proceeds in two steps.
- **Step 1:** Compute the STFT (w.r.t. window $w \in \mathbb{R}^N$ & hop size $H < N$)

$$\mathbf{X}(m, k) := \sum_{n < N} x(n + mH) \cdot w(n) \cdot \exp(-2\pi i n k / N).$$

- Thus $\mathbf{X}(m, k)$ is the k th Fourier coefficient of the m th frame with $m \in \mathbb{Z}$ and $k \in [0 : K]$, where $K = N/2$.
- **Step 2:** Reduce STFT to sparse set of time-frequency points by **peak-picking strategy** (use parameters $\tau > 0$ and $\kappa > 0$): pick all (m_0, k_0) satisfying

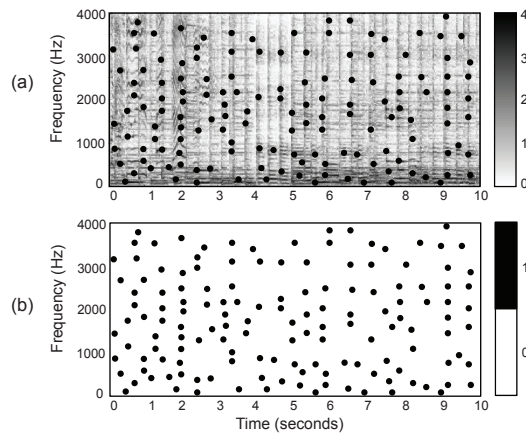
$$|\mathbf{X}(m_0, k_0)| > |\mathbf{X}(m, k)|$$

for all $(m, k) \in [m_0 - \tau : m_0 + \tau] \times [k_0 - \kappa : k_0 + \kappa] \cap \mathbb{Z} \times [0 : K]$.

- Note that increasing the size of the neighborhoods makes it harder for a point to be selected. Thus the parameters τ and κ can be used to adjust the density of the selected peaks.

Illustration of peak selection

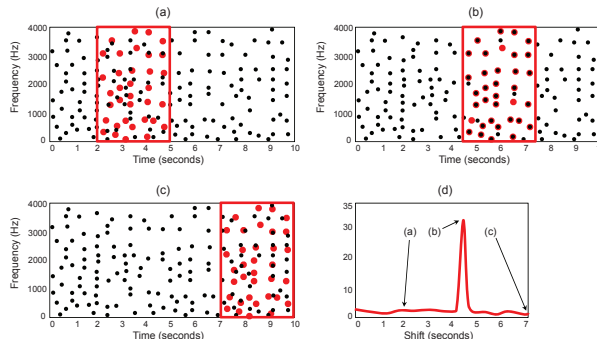
- The following figure illustrates the peak-picking audio fingerprints.
- (a) shows the magnitude spectrogram with extracted peaks.
- (b) shows the resulting **constellation map**, which is the indicator function of all picked time-frequency points (m_0, k_0) .



Fingerprint matching: basic idea

- Consider
 - ◆ $\mathcal{Q}$: a short **query** audio fragment,
 - ◆ $\mathcal{D}$: a **database recording**.
 - ◆ $C(\mathcal{Q})$ and $C(\mathcal{D})$: the corresponding constellation maps.
- **Basic idea of fingerprint matching:**
 - ◆ Put the constellation map of $\mathcal{D}$ on a strip chart.
 - ◆ Put the constellation map of $\mathcal{Q}$ on a transparent piece of plastic.
 - ◆ Shift the latter over the former.
 - ◆ When the number of matching points is significant, the query is considered to be contained in the document.
 - ◆ The proper time offset of the matching position within $C(\mathcal{D})$ is given by the shift.

Illustration of fingerprint matching



- Constellation maps of database document $\mathcal{D}$ (black, small dots) and a query $\mathcal{Q}$ (red, large dots).
- The constellation map of $\mathcal{Q}$ is shifted and locally compared with the one of $\mathcal{D}$.
- **(a)** Shift corresponding to time offset $t = 2$ sec. **(b)** Shift corresponding to time offset $t = 4.5$ sec. (matching position) **(c)** Shift corresponding to time offset $t = 7$ sec. **(d)** Matching function Δ_C , see below.

Formalization of matching procedure

- Recall that a peak position is specified by its coordinates (m, k) consisting of a frame index $m \in \mathbb{Z}$, also called a **time stamp**, and a **frequency stamp** $k \in [0 : K]$. A **constellation map** is then a finite set of such coordinates.
- Thus $C(\mathcal{D})$ and $C(\mathcal{Q})$ are finite subsets of $\mathbb{Z} \times [0 : K]$.
- Shifting the query by $n \in \mathbb{Z}$ positions yields the **shifted constellation map**

$$n + C(\mathcal{Q}) := \{(n + m, k) \mid (m, k) \in C(\mathcal{Q})\}.$$

- To obtain the matching peak coordinates between a shifted query and a database document, we introduce the **matching function** $\Delta_C : \mathbb{Z} \rightarrow \mathbb{N}_0$ by

$$\Delta_C(n) := |(n + C(\mathcal{Q})) \cap C(\mathcal{D})|.$$

- When the query and a database document are unrelated, the number $\Delta_C(n)$ is usually small compared to $|C(\mathcal{Q})|$. Only if the query is contained in the database document, the matching function has a large value $\Delta_C(n)$ for some shift index $n \in \mathbb{Z}$.
- See (d) on the last figure for an illustration of the matching function Δ_C .

Indexing: preliminary remarks

- In the matching process described so far, the query needs to be compared against all sections (having the same duration as the query) of all documents contained in the database.
- Obviously, such an exhaustive search strategy, whose run-time linearly depends on the number and sizes of the documents, is not feasible for large databases containing millions of recordings. (E.g., Shazam's database contains more than 20 million recordings!)
- In view of **scalability**, one requires search strategies that facilitate fast information access without sacrificing accuracy of the retrieval results. Such search strategies typically use **indexing techniques**, which optimize speed and performance by cutting down the search space through suitable look-up operations.
- An **index** is constructed similarly to a traditional book index, which consists of a collection of alphabetically ordered key words. For each key word, there is a list of increasing page numbers indicating the occurrences of the given key word in the book.

Inverted lists: preliminary remarks

- We now introduce a general indexing and retrieval framework based on inverted lists.
- Assume that the data items to be indexed consist of a time stamp $m \in \mathbb{Z}$ as well as a hash $h \in \mathcal{H}$, where $\mathcal{H}$ denotes a finite set of possible hash values.
- In information retrieval, the notion of a **hash** is used to refer to a fixed-length identifier (e.g., a binary string consisting of a fixed number of bits) that acts as a shortened and compact reference to some original, more complex data entity.
- In analogy to the traditional book index, the hash values play the role of the key words and the time stamps the role of the page numbers.
- In the following, we assume that a document $\mathcal{D}$ can be represented by a finite set of **data items**. The resulting set $\mathcal{F}(\mathcal{D}) \subset \mathbb{Z} \times \mathcal{H}$ is also referred to as **feature representation** of $\mathcal{D}$.
- In practice, one has to deal with an entire database containing a large number of documents. To simplify notation, we only regard the case that the **database consists of a single document** $\mathcal{D}$. This can be assumed w.l.o.g. since one may concatenate all database documents to form a single document, and keeping track of possible document boundaries in an additional data structure.

Formalization of index-based retrieval

- Let $\mathcal{F}(\mathcal{D}) \subset \mathbb{Z} \times \mathcal{H}$ denote the feature representation of the whole database $\mathcal{D}$. To every hash $h \in \mathcal{H}$ we define the corresponding **inverted list** $L(h) = L_{\mathcal{D}}(h)$ consisting of all time stamps $m \in \mathbb{Z}$ with $(m, h) \in \mathcal{F}(\mathcal{D})$, where the time stamps are sorted in increasing order:

$$L(h) := \{m \in \mathbb{Z} \mid (m, h) \in \mathcal{F}(\mathcal{D})\} = \{m_1 < m_2 < \dots < m_r\}.$$

Below we will write $L(h)$ as sorted list: $L(h) = (m_1, m_2, \dots, m_r)$.

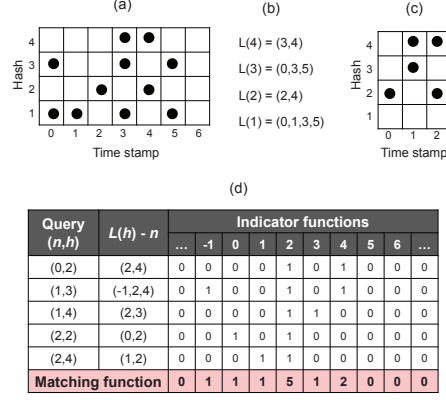
- Let $\mathcal{Q}$ be a query with feature representation $\mathcal{F}(\mathcal{Q})$. For $n \in \mathbb{Z}$, the **n -shifted version** of $\mathcal{F}(\mathcal{Q})$ is defined by

$$n + \mathcal{F}(\mathcal{Q}) := \{(n + m, h) \mid (m, h) \in \mathcal{F}(\mathcal{Q})\}.$$

- The **matching function** $\Delta_{\mathcal{F}} : \mathbb{Z} \rightarrow \mathbb{N}_0$ (w.r.t. $\mathcal{Q}$ and $\mathcal{D}$) is defined by

$$\Delta_{\mathcal{F}}(n) := |(n + \mathcal{F}(\mathcal{Q})) \cap \mathcal{F}(\mathcal{D})|.$$

Illustration of indexed-based retrieval



- (a): feature representation $\mathcal{F}(\mathcal{D}) \subset \mathbb{Z} \times \mathcal{H}$ of database $\mathcal{D}$ with $\mathcal{H} = [1 : 4]$.
- (b): inverted lists $L(h) = L_{\mathcal{D}}(h)$ for $\mathcal{F}(\mathcal{D})$.
- (c): feature representation $\mathcal{F}(\mathcal{Q})$ of query $\mathcal{Q}$.
- (d): computation of the matching function $\Delta_{\mathcal{F}} \stackrel{!}{=} \sum_{(n,h) \in \mathcal{F}(\mathcal{Q})} \mathbf{1}_{L(h)-n}$.

Computation of the matching function

Theorem 1. Given $\mathcal{D}$ and $\mathcal{Q}$, the corresponding matching function $\Delta_{\mathcal{F}} : \mathbb{Z} \rightarrow \mathbb{N}_0$ is a sum of indicator functions:

$$\Delta_{\mathcal{F}} = \sum_{(n,h) \in \mathcal{F}(\mathcal{Q})} \mathbf{1}_{L(h)-n},$$

where $L(h) = L_{\mathcal{D}}(h)$ and $\mathbf{1}_{L(h)-n}(m) = 1$ if $m \in L(h) - n$, else $\mathbf{1}_{L(h)-n}(m) = 0$.

Proof. Note that

$$\mathcal{F}(\mathcal{D}) = \bigsqcup_{h \in \mathcal{H}} L_{\mathcal{D}}(h) \times \{h\} \quad \text{and} \quad \mathcal{F}(\mathcal{Q}) = \bigsqcup_{h \in \mathcal{H}} L_{\mathcal{Q}}(h) \times \{h\}.$$

Thus with $m + \mathcal{F}(\mathcal{Q}) = \bigsqcup_{h \in \mathcal{H}} (m + L_{\mathcal{Q}}(h)) \times \{h\}$ we obtain for $m \in \mathbb{Z}$

$$\begin{aligned} \Delta_{\mathcal{F}}(m) &= |(m + \mathcal{F}(\mathcal{Q})) \cap \mathcal{F}(\mathcal{D})| = \sum_{h \in \mathcal{H}} |(m + L_{\mathcal{Q}}(h)) \cap L_{\mathcal{D}}(h)| \\ &= \sum_{h \in \mathcal{H}} |L_{\mathcal{Q}}(h) \cap (L_{\mathcal{D}}(h) - m)| = \sum_{(n,h) \in \mathcal{F}(\mathcal{Q})} \mathbf{1}_{L_{\mathcal{D}}(h)-n}(m). \quad \square \end{aligned}$$

Complexity of computing matching functions

- The inverted lists $L_{\mathcal{D}}(h)$, $h \in \mathcal{H}$, do not depend on the query, thus computing all $L_{\mathcal{D}}(h)$ can be done **offline** and is not part of our complexity analysis.
- First note that the number $|\mathcal{F}(\mathcal{D})|$ of database items is in general very large, whereas the number $|\mathcal{F}(\mathcal{Q})|$ of query items is small.
- Assuming that the hash values of the database items are more or less evenly distributed over the set $\mathcal{H}$ of hashes, each of the inverted lists $L_{\mathcal{D}}(h)$, $h \in \mathcal{H}$, contains roughly $|\mathcal{F}(\mathcal{D})|/|\mathcal{H}|$ elements.
- In the query stage, only the information contained in the inverted lists $L_{\mathcal{D}}(h)$ corresponding to the elements (n, h) of $\mathcal{F}(\mathcal{Q})$ is needed.
- These $|\mathcal{F}(\mathcal{Q})|$ lists need to be accessed, shifted, and further processed to derive the matching functions—all operations that are linear in the sizes of the required lists. Therefore, the **overall complexity in processing a query is linear in**

$$\boxed{\frac{|\mathcal{F}(\mathcal{Q})| \cdot |\mathcal{F}(\mathcal{D})|}{|\mathcal{H}|}}. \quad (1)$$

Gain of indexed-based computation

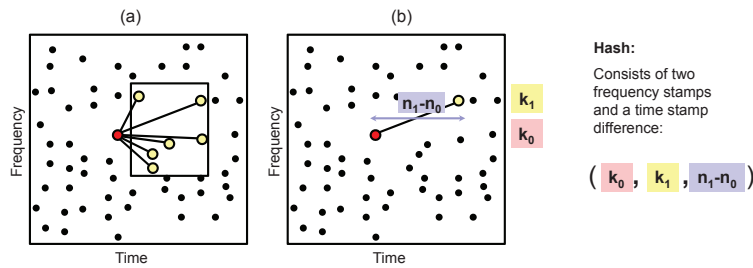
- Let us analyze what we have gained by using the index-based computation of the matching function.
- Compared to the naive approach for computing the matching function, which requires $|\mathcal{F}(\mathcal{Q})| \cdot |\mathcal{F}(\mathcal{D})|$ operations, we have gained an increase in efficiency by a factor of $|\mathcal{H}|$.
- Even though for a fixed number of hashes, the overall complexity for processing a query is still linear in the size $|\mathcal{F}(\mathcal{D})|$ of the database items, the speed-up in practical applications can be drastic in the case $|\mathcal{H}|$ is large.
- Therefore, in order to speed-up the computations, one straightforward idea is to increase the number of hashes.
- However, this comes at a cost—as we will see below when applying the indexed-based retrieval concept for audio identification.

Indexed-based audio identification

- The above indexing and retrieval framework can be directly applied to our fingerprinting scenario based on spectral peaks.
- As for the feature representations, we use the constellation maps $\mathcal{F}(\mathcal{Q}) = \mathcal{C}(\mathcal{Q})$ for the query and $\mathcal{F}(\mathcal{D}) = \mathcal{C}(\mathcal{D})$ for the database document.
- The set of possible hashes is then given by $\mathcal{H} = [0 : K]$, which consists of the different frequency stamps resulting from the STFT computation.
- With these settings, the matching function is given by $\Delta_{\mathcal{C}} = \Delta_{\mathcal{F}}$.
- This simple indexing approach turns out to be **problematic**, since the number of frequency stamps may be too small to sufficiently speed up the retrieval process:
 - ◆ For example, using a 1024-bin frequency axis yields only a speed-up factor of 1024 compared to the brute force approach. In view of the huge databases containing millions of recordings, much larger speed-up factors are needed.
 - ◆ One idea could be to increase the number of hashes by using a finer frequency resolution in the peak computation, e.g., using $2^{14} = 16384$ frequency bins instead of $2^{10} = 1024$.
 - ◆ This, however, would drastically affect the **robustness** of the overall system: spectral distortions in the query would lead to modifications of the frequency stamps of spectral peaks resulting in many **mismatches**.

Improved hashing via pairs of peaks

- How can one increase the specificity and the number of hashes without sacrificing the robustness of the fingerprints? We describe a possible approach (Wang, 2003).
- **Main idea:** form fingerprint hashes by considering **pairs of spectral peaks**.



- Fix an **anchor point** $(n_0, k_0) \in \mathcal{C}(\mathcal{D})$ (the red point in (a)) and associate to it all spectral peak positions $(n_1, k_1) \in \mathcal{C}(\mathcal{D})$ in a rectangular **target zone** $T_{(n_0, k_0)}$ of **width** W (these are the yellow points in (a)).
- Every $((n_0, k_0), (n_1, k_1)) \in \mathcal{C}(\mathcal{D}) \times (T_{(n_0, k_0)} \cap \mathcal{C}(\mathcal{D}))$ yields a **new hash**:

$$(k_0, k_1, n_1 - n_0) \in [0 : K] \times [0 : K] \times [1 : W], \text{ see (b).}$$

A new feature representation

- Instead of $\mathcal{H} = [0 : K]$, the revised finite set of possible hashes is now

$$\mathcal{H}_3 := [0 : K] \times [0 : K] \times [1 : W] .$$

- Based on such triples as hashes, we redefine our feature representations.
- For every $(n_0, k_0) \in \mathcal{C}(\mathcal{D})$ we introduce the set of corresponding hashes:

$$\mathcal{T}(n_0, k_0) := \{(k_0, k_1, n_1 - n_0) \mid (n_1, k_1) \in T_{(n_0, k_0)} \cap \mathcal{C}(\mathcal{D})\} \subset \mathcal{H}_3$$

- A new feature representation $\mathcal{F}_3(\mathcal{D})$ for $\mathcal{D}$ is obtained by adding the time stamp n_0 of the anchor point (n_0, k_0) to each of these triples and considering **every** point of $\mathcal{C}(\mathcal{D})$ as an anchor:

$$\mathcal{F}_3(\mathcal{D}) := \bigcup_{(n_0, k_0) \in \mathcal{C}(\mathcal{D})} \{n_0\} \times \mathcal{T}(n_0, k_0) .$$

- Similarly, one obtains a new feature representation $\mathcal{F}_3(\mathcal{Q})$ for the query $\mathcal{Q}$.

Audio identification revisited

- With these new feature representations, we can use exactly the same indexing and retrieval framework as above for computing the **new matching function** $\Delta_{\mathcal{F}_3} : \mathbb{Z} \rightarrow \mathbb{N}_0$, which, for $n \in \mathbb{Z}$, is defined by

$$\Delta_{\mathcal{F}_3}(n) := |(n + \mathcal{F}_3(\mathcal{Q})) \cap \mathcal{F}_3(\mathcal{D})| .$$

Here, $n + \mathcal{F}_3(\mathcal{Q}) := \{(n + m, h) \mid (m, h) \in \mathcal{F}_3(\mathcal{Q})\}$.

- With the **new inverted lists** $L_3(h) := \{m \in \mathbb{Z} \mid (m, h) \in \mathcal{F}_3(\mathcal{D})\}$ for each $h \in \mathcal{H}_3$, the new matching function is also a sum of indicator functions, see Theorem 1 on Slide 17:

$$\Delta_{\mathcal{F}_3} = \sum_{(n, h) \in \mathcal{F}_3(\mathcal{Q})} \mathbf{1}_{L_3(h) - n} .$$

- Still, a high value $\Delta_{\mathcal{F}_3}(m)$ is a good indicator that the query is likely to be part of the database document.

Storage requirements of the new approach

- Our **aim** was to **accelerate the retrieval process** by reducing the length of the inverted lists **without loosing fingerprint robustness**. To this end we increased the **specificity of the fingerprints** by considering pairs of spectral peak positions along with their distance instead of individual peaks.
- Obviously, the number of items to be indexed and queried increases by this construction. Assuming that the peak coordinates are more or less evenly distributed in a constellation map and using suitably shifted target zones of the same fixed size for all anchor points, the number of constellation points per target zone is roughly the same. We call this number the **fan-out** of the target zone and denote it by $F \in \mathbb{N}$.
- **Example:** if $F = 10$, then the number of items (the pairs of peaks) contained in the feature representation $\mathcal{F}_3(\mathcal{D})$ is approximately ten times the number of original items (per peak) in the original constellation map $\mathcal{C}(\mathcal{D})$. The same applies to $\mathcal{F}_3(\mathcal{Q})$.
- Thus the fan-out factor leads directly to a cost factor in terms of the number of data items to be stored, indexed, and processed.

Speed-up of the new approach

- Despite the above mentioned increase in data items, the new approach gains a tremendous acceleration in the search process.
- Let us assume that each entry in every triple $(k_0, k_1, n_1 - n_0) \in \mathcal{H}_3$ can be encoded using B bits. Then 2^{B+B+B} is the number of possible hash values, as opposed to the number of 2^B hash values when using a single frequency stamp. So the **specificity** of the hashes has increased by a factor of 2^{B+B} .
- On the downside, the new feature representations need more storage: $|\mathcal{F}_3(\mathcal{D})| \approx F \cdot |\mathcal{F}(\mathcal{D})|$ and $|\mathcal{F}_3(\mathcal{Q})| \approx F \cdot |\mathcal{F}(\mathcal{Q})|$.
- With Equation (1) on Slide 18 we obtain for the **complexity of query processing** with the new approach:

$$\frac{|\mathcal{F}_3(\mathcal{Q})| \cdot |\mathcal{F}_3(\mathcal{D})|}{|\mathcal{H}_3|} \approx \frac{F \cdot |\mathcal{F}(\mathcal{Q})| \cdot F \cdot |\mathcal{F}(\mathcal{D})|}{|\mathcal{H}|^3} = \frac{F^2}{|\mathcal{H}|^2} \cdot \frac{|\mathcal{F}(\mathcal{Q})| \cdot |\mathcal{F}(\mathcal{D})|}{|\mathcal{H}|}.$$

Thus the time to process a query has been reduced by a factor of $|\mathcal{H}|^2/F^2$.

- **Example:** Having $F = 10$ and using $B = 10$ bits, this results in a huge speed-up by a factor of $2^{20}/100 \approx 10000$.

Robustness of the new approach (1)

- First note that we have not changed the way the spectral peaks are computed. We only combined two peaks to form a new, more complex and hence more specific item. However, being a combination of two individual peaks, it becomes more likely that this item will be modified due to **signal distortions**.
- To be more specific, let us assume that the **probability of a spectral peak to survive the distortions** the query audio fragment may be exposed to is given by a number $p \in [0, 1]$. For example, $p = 0.2$ means that 20 % of the original peaks survive. Furthermore, let us simplistically assume that this probability is **independent** and **identically distributed** over all points involved. Then the probability that two specific points survive is p^2 . For example, in the case $p = 0.2$, one obtains that this probability is only $p^2 = 0.04$.
- Assuming that the **distance between two surviving peaks is not affected by the distortion**, the probability of a triple $(k_0, k_1, n_1 - n_0)$ to survive is also p^2 . This reduction in item resilience is a tradeoff against the tremendous amount of speed-up provided.

Robustness of the new approach (2)

- The reduced probability of individual item survival is mitigated by the F times larger number of available items compared to the number of original constellation points. For a given anchor point, the probability of at least one item surviving is the joint probability of the anchor point and at least one target point in its target zone surviving. This probability is given by

$$p \cdot (1 - (1 - p)^F).$$

(Details will be discussed in the Exercises.)

- **Example:** if $F = 10$ and $p = 0.2$, the probability that at least one target point surviving is

$$1 - (1 - p)^{10} = 1 - 0.8^{10} \approx 0.9.$$

Therefore, we are not much worse off than before.

- In summary, we have seen that by using more complex items for indexing, we have traded off approximately $F = 10$ times the storage space for approximately 10000 times improvement in speed, and a small loss in probability of audio identification.

Further applications *

We mention further applications of audio identification techniques:

- **Broadcast monitoring:** monitor radio stations, e.g., to ensure that advertisements were actually broadcast, to automatically create playlists, or compile charts.
- **Copyright monitoring:** a prominent example is YouTube's **Content ID**. In order to check for infringements, YouTube allows copyright holders to upload audio or video reference files, and if a match is found, the infringement is either (a) tolerated, (b) the copyright holder receives compensation, or (c) the content is blocked. Which of the three routes is chosen is up to the copyright holder.
- **File metadata correction:** Often, when ripped from CD without the use of a service like Gracenote's CDDb or freeDB, audio recordings are given semi-anonymous names like `Track01.mp3` without any additional information. To fix this issue, metadata services identify badly labeled tracks and import or update the missing or wrong metadata.

References *

To a large extent this chapter is based on Section 7.1 of the very recommendable book:

- **Meinard Müller: Fundamentals of Music Processing, Springer 2015.**

The Shazam Technology is based on the following paper:

- Avery Li-Chun Wang: An Industrial-Strength Audio Search Algorithm, ISMIR, 2003, pp. 7–13.

For alternative approaches to audio identification we refer to

- Jaap Haitsma & Ton Kalker: A Highly Robust Audio Fingerprinting System, ISMIR, 2002, pp. 107–115.
- Michael Clausen & Frank Kurth: A unified approach to content-based and fault tolerant music identification, IEEE Transactions on Multimedia, 6, (2004), pp. 717–731.